



SH320 Data Engineering

Spring 2026 Semester

7 Working with Time Series

Hyun Ahn

School of Computing and Artificial Intelligence

hyunahn@hs.ac.kr

Core Libraries for Data Science



- **[W2] N-Dimensional Arrays**
 - NumPy arrays
 - Array manipulation
- **[W3] Efficient Computation in NumPy**
 - Universal functions
 - Advanced features

- **[W5] Data Manipulation: Part 1**
 - Pandas objects: Series, DataFrame
 - Implicit / explicit indexing
- **[W6] Data Manipulation: Part 2**
 - Pandas objects & manipulation
 - Combining datasets
 - Exploratory data analysis

Data Engineering



- **[W7] Data Acquisition**
 - Web scraping
 - RESTful API

- **[W9] Working with Time Series**
 - Time series manipulation
 - Time-series-specific operations

7.1 시계열 데이터 생성

What Is Time Series?

- **시계열** Time Series: 어떤 관측결과를 **시간 축에 따라 계열화한 데이터**



그림. 시계열 분석 응용들 (Source: DataScienceLearn)

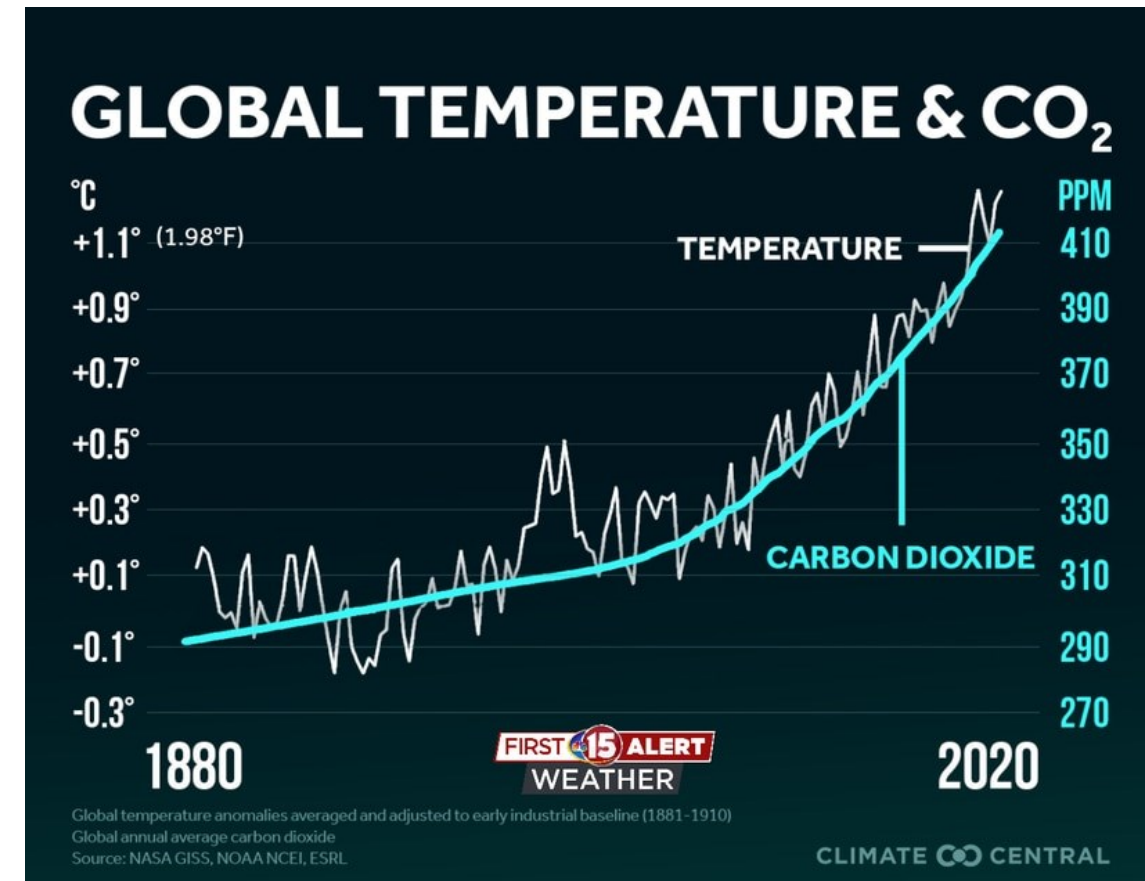


그림. 세상에서 가장 중요한 시계열 (Source: NASA)

Working with Time Series in Python



- Pandas 의 개발 목적: 금융 데이터 모델링 ⇨ 시계열 처리 관련 기능 강력
- 주요 개념
 - 타임스탬프^{Timestamp}: 특정 시점(예: 2021-10-27 07:00:00)
 - 시간 간격^{Interval}: 두 시점 사이의 시간 구간
 - 기간^{Period}: 특정 시작일/종료일로 표현되는 시간 구간(예: 2021-10-01~2021-10-31)
 - 지속기간^{Duration}: 시간 구간의 길이

Representations of Time Series in Python



- 시간 데이터 관련 라이브러리 및 함수
 - Python: datetime, dateutil
 - NumPy: datetime64()
 - Pandas: to_datetime(), to_timedelta()

Dates and Times in Native Python



- datetime, dateutil: 시계열 처리를 위한 파이썬 내장 모듈

- datetime 날짜 정의

```
from datetime import datetime
datetime(year=2025, month=5, day=7)

2025-05-07 00:00:00
```

- 문자열 파싱을 통한 날짜 해석

```
from dateutil import parser
date = parser.parse("7th of May, 2025")

2025-05-07 00:00:00
```

- datetime 요일 출력

- strftime() 사용법 ([link](#))

```
date.strftime(`%A`)

'Wednesday'
```

Datetime Arrays: NumPy's datetime64

- NumPy의 datetime64: 배열 형태로 Datetime 처리할 때 적합(예: 벡터화)
 - datetime64 객체 생성: `np.array()` 또는 `np.datetime64()`

```
import numpy as np
date = np.array(['2025-05-07', '2025-05-08'], dtype=np.datetime64)
```

```
date = np.datetime64('2025-05-01')
```

- 벡터화 연산을 통한 Datetime 배열 생성

```
date + np.arange(31)

array(['2025-05-01', '2025-05-02', '2025-05-03', '2025-05-04',
      ...,
      '2025-05-29', '2025-05-30', '2025-05-31'], dtype='datetime64[D]')
```


Types Arrays of Times: NumPy's datetime64



- 시간 정밀도 설정

- 분 단위의 datetime64

```
t = np.datetime64('2025-05-07 14:30')
```

```
2025-05-07T14:30
```

- 정밀도를 나노초 Nanosecond 로 설정

```
t = np.datetime64('2025-05-07 14:30:59.59', 'ns')
```

```
2025-05-07T14:30:59.990000000
```

NumPy datetime64 Codes



인코딩 가능한 시간 범위

Code	Meaning	Time span (relative)	Time span (absolute)
Y	Year	$\pm 9.2\text{e}18$ years	[9.2e18 BC, 9.2e18 AD]
M	Month	$\pm 7.6\text{e}17$ years	[7.6e17 BC, 7.6e17 AD]
W	Week	$\pm 1.7\text{e}17$ years	[1.7e17 BC, 1.7e17 AD]
D	Day	$\pm 2.5\text{e}16$ years	[2.5e16 BC, 2.5e16 AD]
h	Hour	$\pm 1.0\text{e}15$ years	[1.0e15 BC, 1.0e15 AD]
m	Minute	$\pm 1.7\text{e}13$ years	[1.7e13 BC, 1.7e13 AD]
s	Second	$\pm 2.9\text{e}12$ years	[2.9e9 BC, 2.9e9 AD]
ms	Millisecond	$\pm 2.9\text{e}9$ years	[2.9e6 BC, 2.9e6 AD]
us	Microsecond	$\pm 2.9\text{e}6$ years	[290301 BC, 294241 AD]
ns	Nanosecond	± 292 years	[1678 AD, 2262 AD]
ps	Picosecond	± 106 days	[1969 AD, 1970 AD]
fs	Femtosecond	± 2.6 hours	[1969 AD, 1970 AD]
as	Attosecond	± 9.2 seconds	[1969 AD, 1970 AD]

Dates and Times in Pandas



- **Pandas** 에서의 날짜/시간 처리

- 사용 편의성(datetime, dateutil), 배열 처리(datetime64) 기능 제공

```
import pandas as pd
date = pd.to_datetime("1985-09-18")
```

```
1985-09-18 00:00:00
```

```
date.strftime('%A')
```

```
Wednesday
```

```
date + pd.to_timedelta(np.arange(7))
```

```
DatetimeIndex([
    '2021-10-27 00:00:00',
    '2021-10-27 00:00:00.000000001',
    '2021-10-27 00:00:00.000000002',
    '2021-10-27 00:00:00.000000003',
    '2021-10-27 00:00:00.000000004',
    '2021-10-27 00:00:00.000000005',
    '2021-10-27 00:00:00.000000006'],
    dtype='datetime64[ns]', freq=None)
```

Indexing by Time



- 타임스탬프를 인덱스로 사용

```
ind = pd.DatetimeIndex(['2022-10-27', '2022-10-28', '2022-10-29', '2022-10-30',  
                        '2022-10-31', '2022-11-01', '2022-11-02'])
```

```
ser = pd.Series([0, 1, 2, 3, 4, 5, 6], index=ind)
```

```
2022-10-27    0  
2022-10-28    1  
2022-10-29    2  
2022-10-30    3  
2022-10-31    4  
2022-11-01    5  
2022-11-02    6  
dtype: int64
```

Indexing by Time



- 인덱싱 예제

```
ser[ '2022-10-27' : '2022-10-31' ]
```

```
2022-10-27    0
2022-10-28    1
2022-10-29    2
2022-10-30    3
2022-10-31    4
dtype: int64
```

```
ser[ '2022-11' ]
```

```
2022-11-01    5
2022-11-02    6
dtype: int64
```

Time Series Data Structures



- 시계열 데이터를 위한 **Pandas** 데이터 구조
 - 타임스탬프: `Timestamp`, `DatetimeIndex`
 - 기간: `Period`, `PeriodIndex`
 - 지속기간: `Timedelta`, `TimedeltaIndex`

Timestamp and DatetimeIndex



- Timestamp, DatetimeIndex: 날짜/시간을 표현하는 기본적인 객체
 - Timestamp 를 여러 개 저장할 경우 DatetimeIndex 객체로 저장

```
dates = pd.to_datetime([datetime(2015, 7, 3), "4th of July, 2015", '2015-Jul-6',  
                        '07-07-2015', '20150708'])
```

```
DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-06', '2015-07-07', '2015-07-08'],  
              dtype='datetime64[ns]', freq=None)
```

PeriodIndex



- `to_period()`: DatetimeIndex $\Rightarrow$ PeriodIndex 변환

```
dates.to_period('D')
```

```
PeriodIndex(['2015-07-03', '2015-07-04', '2015-07-06', '2015-07-07', '2015-07-08'],  
            dtype='period[D]', freq='D')
```

```
dates.to_period('W')
```

```
PeriodIndex(['2015-06-29/2015-07-05', '2015-06-29/2015-07-05',  
            '2015-07-06/2015-07-12', '2015-07-06/2015-07-12',  
            '2015-07-06/2015-07-12'],  
            dtype='period[W-SUN]', freq='W-SUN')
```


TimedeltaIndex



- TimedeltaIndex: 타임스탬프간 뺄셈 연산을 통해 생성(= 시간차)

```
dates - dates[0]
```

```
TimedeltaIndex(['0 days', '1 days', '3 days', '4 days', '5 days'],  
dtype='timedelta64[ns]', freq=None)
```

Regular Sequences

- Q: 만약 타임스탬프 없이 값으로만 구성된 데이터를 분석해야 된다면?

- 빈도 기반으로 정규 시퀀스 생성하기

- 타임스탬프: `pd.date_range()`
- 기간: `pd.period_range()`
- 지속기간: `pd.timedelta_range()`



pd.date_range()



- 시작/종료 타임스탬프를 이용한 시퀀스 생성

```
pd.date_range('2021-10-27', '2021-10-31')  
  
DatetimeIndex(['2021-10-27', '2021-10-28', '2021-10-29',  
              '2021-10-30', '2021-10-31'], dtype='datetime64[ns]', freq='D')
```

- 시작 타임스탬프, 기간 길이를 이용

```
pd.date_range('2021-10-27', periods=5)  
  
DatetimeIndex(['2021-10-27', '2021-10-28', '2021-10-29',  
              '2021-10-30', '2021-10-31'], dtype='datetime64[ns]', freq='D')
```

pd.date_range()



- 빈도 설정: 원하는 시간 단위의 시퀀스 생성

```
pd.date_range('2021-10-27', periods=12, freq='H')
```

```
DatetimeIndex(['2021-10-27 00:00:00', '2021-10-27 01:00:00',  
              '2021-10-27 02:00:00', '2021-10-27 03:00:00',  
              '2021-10-27 04:00:00', '2021-10-27 05:00:00',  
              '2021-10-27 06:00:00', '2021-10-27 07:00:00',  
              '2021-10-27 08:00:00', '2021-10-27 09:00:00',  
              '2021-10-27 10:00:00', '2021-10-27 11:00:00'],  
              dtype='datetime64[ns]', freq='H')
```

period_range() and timedelta_range()



- 특정 기간에 대한 정규 시퀀스

```
pd.period_range('2021-10', periods=12, freq='M')
```

```
PeriodIndex(['2021-10', '2021-11', '2021-12', '2022-01', '2022-02', '2022-03',  
            '2022-04', '2022-05', '2022-06', '2022-07', '2022-08', '2022-09'],  
            dtype='period[M]', freq='M')
```

- 지속기간에 대한 정규 시퀀스

```
pd.timedelta_range(0, periods=10, freq='H')
```

```
TimedeltaIndex(['00:00:00', '01:00:00', '02:00:00', '03:00:00', '04:00:00',  
               '05:00:00', '06:00:00', '07:00:00', '08:00:00', '09:00:00'],  
               dtype='timedelta64[ns]', freq='H')
```

Frequencies and Offsets



- 세부적인 빈도 및 오프셋 설정 ⇨ 더욱 정밀한 정규 시퀀스 생성 가능

표. Pandas 에서 지원하는 빈도 유형

코드	설명	코드	설명
D	달력상 일	B	영업일
W	주		
M	월말	BM	영업일 기준 월말
Q	분기말	BQ	영업일 기준 분기말
A	연말	BA	영업일 기준 연말
H	시간	BH	영업 시간
T	분		
S	초		
L	밀리초		
U	마이크로초		
N	나노초		

Frequencies and Offsets



– 예: 월말

```
pd.date_range('2021-11-01', periods=10, freq="M")  
  
DatetimeIndex(['2021-11-30', '2021-12-31', '2022-01-31', '2022-02-28',  
              '2022-03-31', '2022-04-30', '2022-05-31', '2022-06-30',  
              '2022-07-31', '2022-08-31'], dtype='datetime64[ns]', freq='M')
```

– 예: 영업일 기준 월말

```
pd.date_range('2021-11-01', periods=10, freq="BM")  
  
DatetimeIndex(['2021-11-30', '2021-12-31', '2022-01-31', '2022-02-28',  
              '2022-03-31', '2022-04-29', '2022-05-31', '2022-06-30',  
              '2022-07-29', '2022-08-31'], dtype='datetime64[ns]', freq='BM')
```

Frequencies and Offsets

- 빈도 코드에 접미사 'S' 추가: 종료 시점 ⇨ 시작 시점

```
pd.date_range('2021-11-01', periods=10, freq="MS")
```

```
DatetimeIndex(['2021-11-01', '2021-12-01', '2022-01-01', '2022-02-01',  
               '2022-03-01', '2022-04-01', '2022-05-01', '2022-06-01',  
               '2022-07-01', '2022-08-01'], dtype='datetime64[ns]', freq='MS')
```

코드	설명
MS	월초
BMS	영업일 기준 월초
QS	분기초
BQS	영업일 기준 분기초
AS	연초
BAS	영업일 기준 연초

표. 시작 시점 관련 빈도 코드

Frequencies and Offsets



– 빈도 + 숫자 결합

```
pd.date_range('2021-11-01', periods=9, freq='2H30T')  
  
DatetimeIndex(['2021-11-01 00:00:00', '2021-11-01 02:30:00',  
              '2021-11-01 05:00:00', '2021-11-01 07:30:00',  
              '2021-11-01 10:00:00', '2021-11-01 12:30:00',  
              '2021-11-01 15:00:00', '2021-11-01 17:30:00',  
              '2021-11-01 20:00:00'], dtype='datetime64[ns]', freq='150T')
```

Frequencies and Offsets

- 오프셋을 이용한 타임스탬프 계산

- 예: 현시점 기준으로 40시간 뒤의 시간은?

```
from pandas.tseries.offsets import DateOffset
ts = pd.Timestamp('2021-10-31 23:28:30')
ts + DateOffset(hours=40)

Timestamp('2021-11-02 15:28:30')
```

- 예: 현시점 기준으로 71년 4 개월 5 일 19 시간 28분 30초 전의 시간은?

```
ts - DateOffset(years=71, months=4, days=5, hours=19, minutes=28,
seconds=30)

Timestamp('1950-06-25 04:00:00')
```

7.2 시계열 전용 연산

Time Series-specific Operations



- pandas 에서 제공하는 시계열 전용 연산
 - 리샘플링^{Resampling} : 시계열 빈도 변경
 - 시프팅^{Shifting} : 시계열 이동
 - 윈도우잉^{Windowing} : 시계열 집계

Time Series-specific Operations



- 예제: 주가^{Stock Price} 시계열
- 필요한 라이브러리
 - `yfinance` : Yahoo Finance 데이터의 빠른 다운로드를 지원

Loading the Google's Price Data

- Yahoo Finance 로부터 구글 주가 데이터 불러오기

```
import yfinance as yf
```

```
goog = yf.download('GOOG', start='2004-01-01', end='2024-12-31')  
goog.head()
```

Date	High	Low	Open	Close	Volume	Adj Close
2004-08-19	51.835709	47.800831	49.813290	49.982655	44871361.0	49.982655
2004-08-20	54.336334	50.062355	50.316402	53.952770	22942874.0	53.952770
2004-08-23	56.528118	54.321388	55.168217	54.495735	18342897.0	54.495735
2004-08-24	55.591629	51.591621	55.412300	52.239197	15319808.0	52.239197
2004-08-25	53.798351	51.746044	52.284027	52.802086	9232276.0	52.802086

ex) Google's Closing Prices

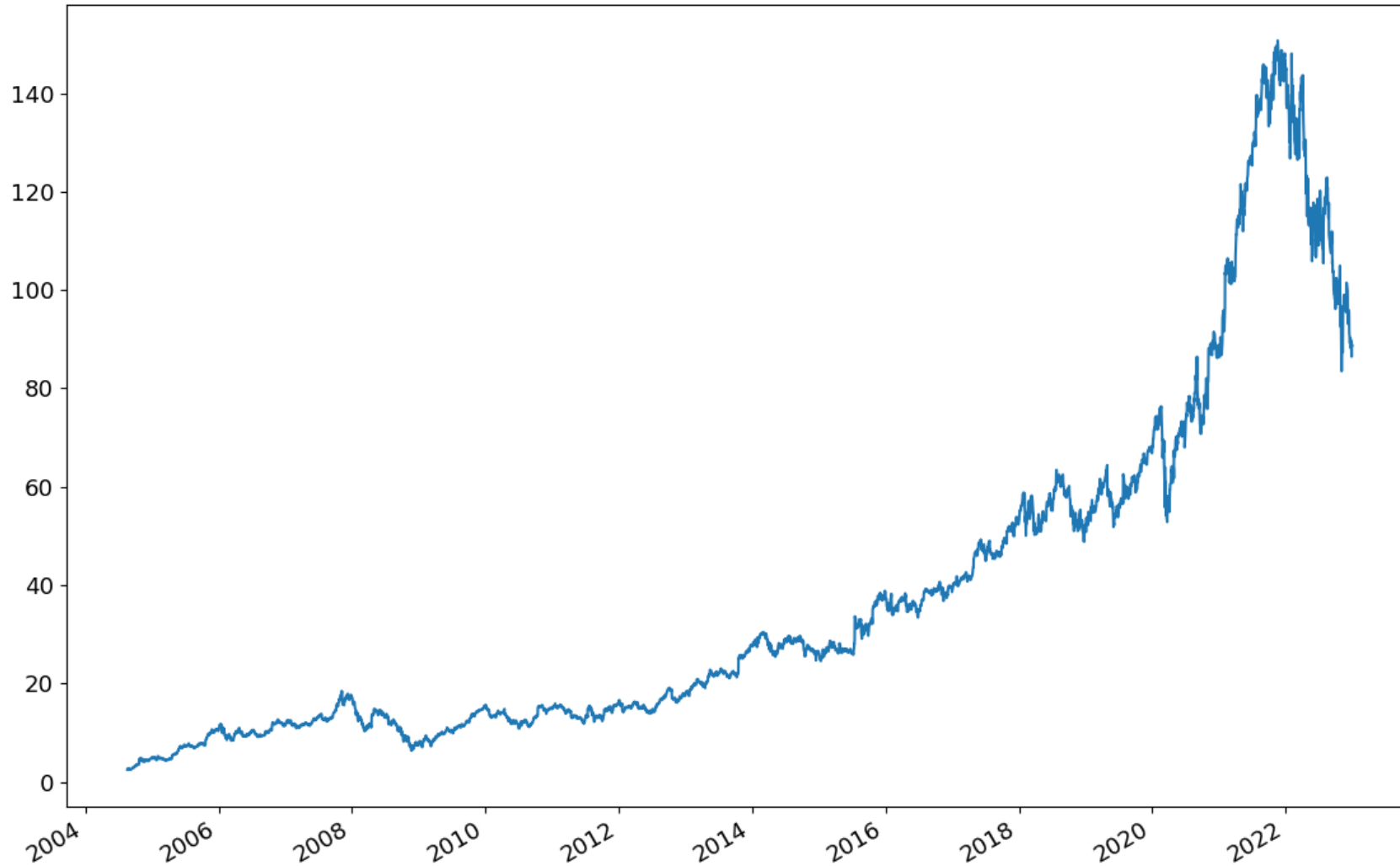


그림. 구글의 주가 데이터(2004년~2022
년)

Resampling

- **리샘플링**: 데이터 표본을 재추출 해서 시계열의 빈도를 변경하는 방법
 - 업샘플링 Upsampling : 시계열 빈도 증가(예: 일별 데이터 ⇨ 시간별 데이터)
 - 다운샘플링 Downsampling : 시계열 빈도 감소(예: 시간별 데이터 ⇨ 일별 데이터)
 - 계산비용, 예측 문제 등에 따라 일반적으로 다운샘플링이 더 많이 활용됨

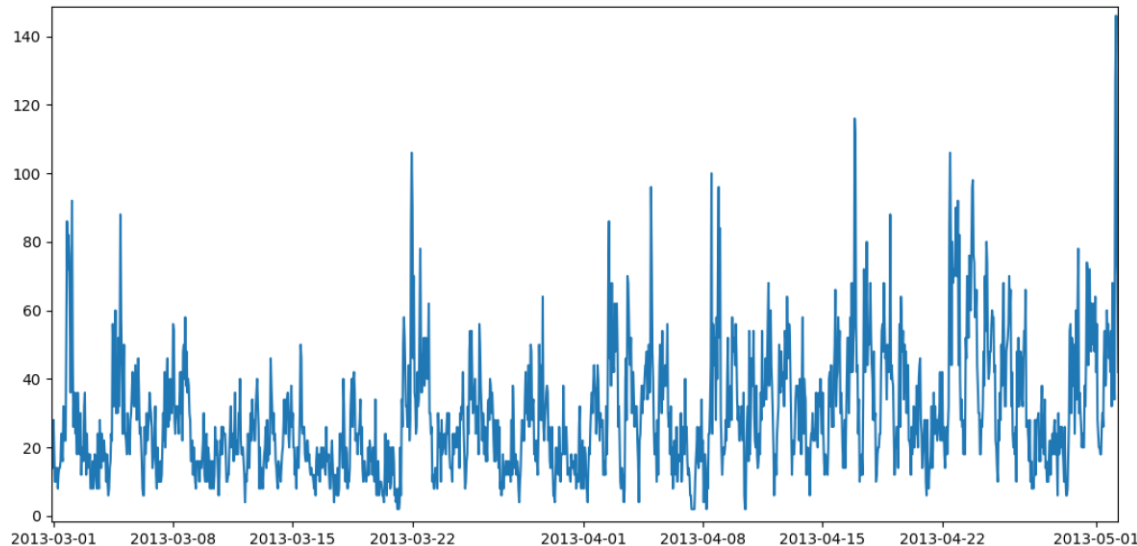


그림. 원본 시계열(시간 별)

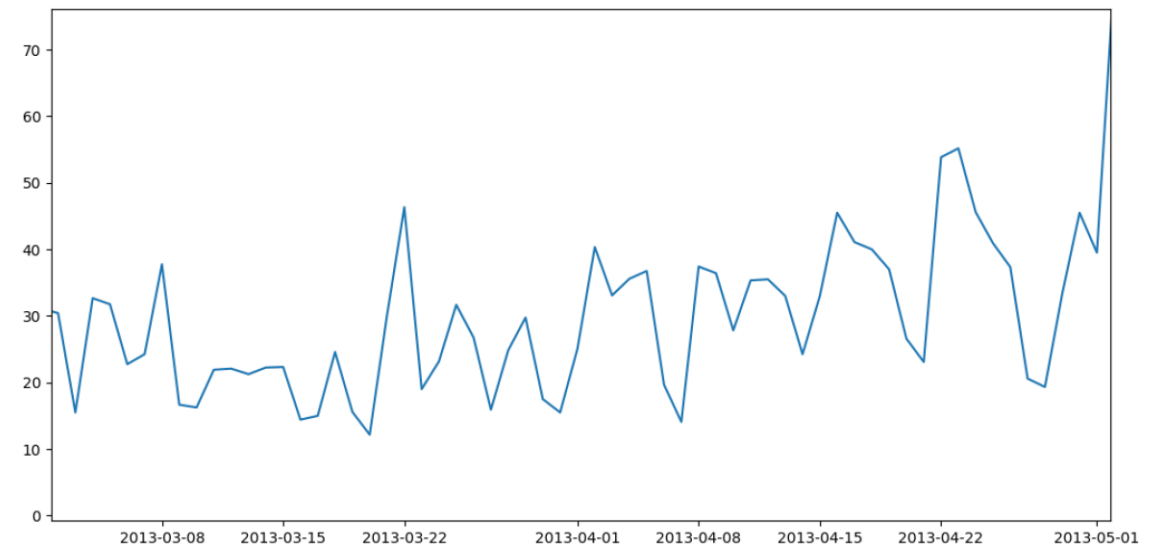
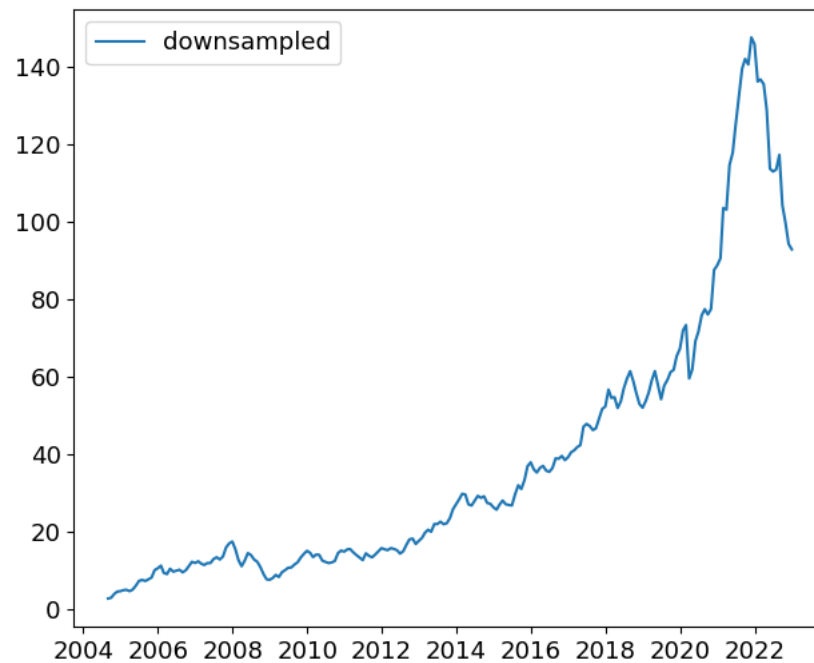
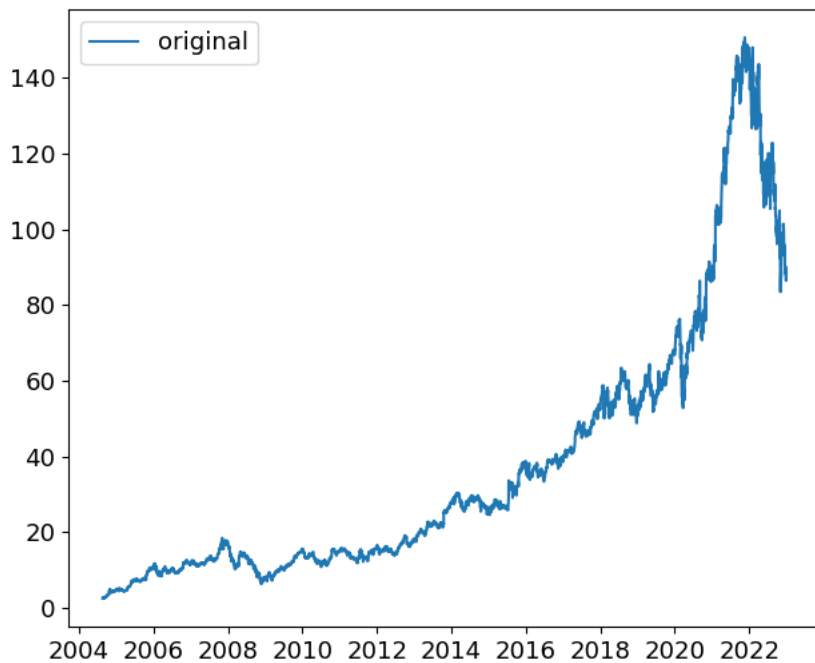


그림. 다운 샘플링 처리된 시계열(일별)

Resampling

- `resample()`: 시계열을 정해진 빈도로 그룹화 및 집계 연산 적용
 - 다운샘플링 예: 일별 주가 $\Rightarrow$ 월별 평균 주가

```
goog.resample('ME').mean().plot()
```



Resampling

- `asfreq()`: 주어진 빈도에 해당되는 데이터 포인트를 선택
 - 기존 데이터 포인트가 제거되어 **NaN** Not as a Number 값이 생성됨
 - 예: 다운샘플링 일별 주가 ⇨ 월별 말일 주가

```
goog.asfreq('ME').plot()
```



Date	
2004-08-31	2.549693
2004-09-30	3.227901
2004-10-31	NaN
2004-11-30	4.532511
2004-12-31	4.801752
...	
2022-07-31	NaN
2022-08-31	109.150002
2022-09-30	96.150002
2022-10-31	94.660004
2022-11-30	101.449997

Resampling



- 업샘플링

- 기존 데이터에서 빈도가 증가하므로 NaN 값이 발생
- NaN 값에 대한 처리(데이터 대체)가 필요함
- 예: 업샘플링 일별 주가 ➔ 12시간 별 평균 주가

```
goog.resample ('12h').mean()
```

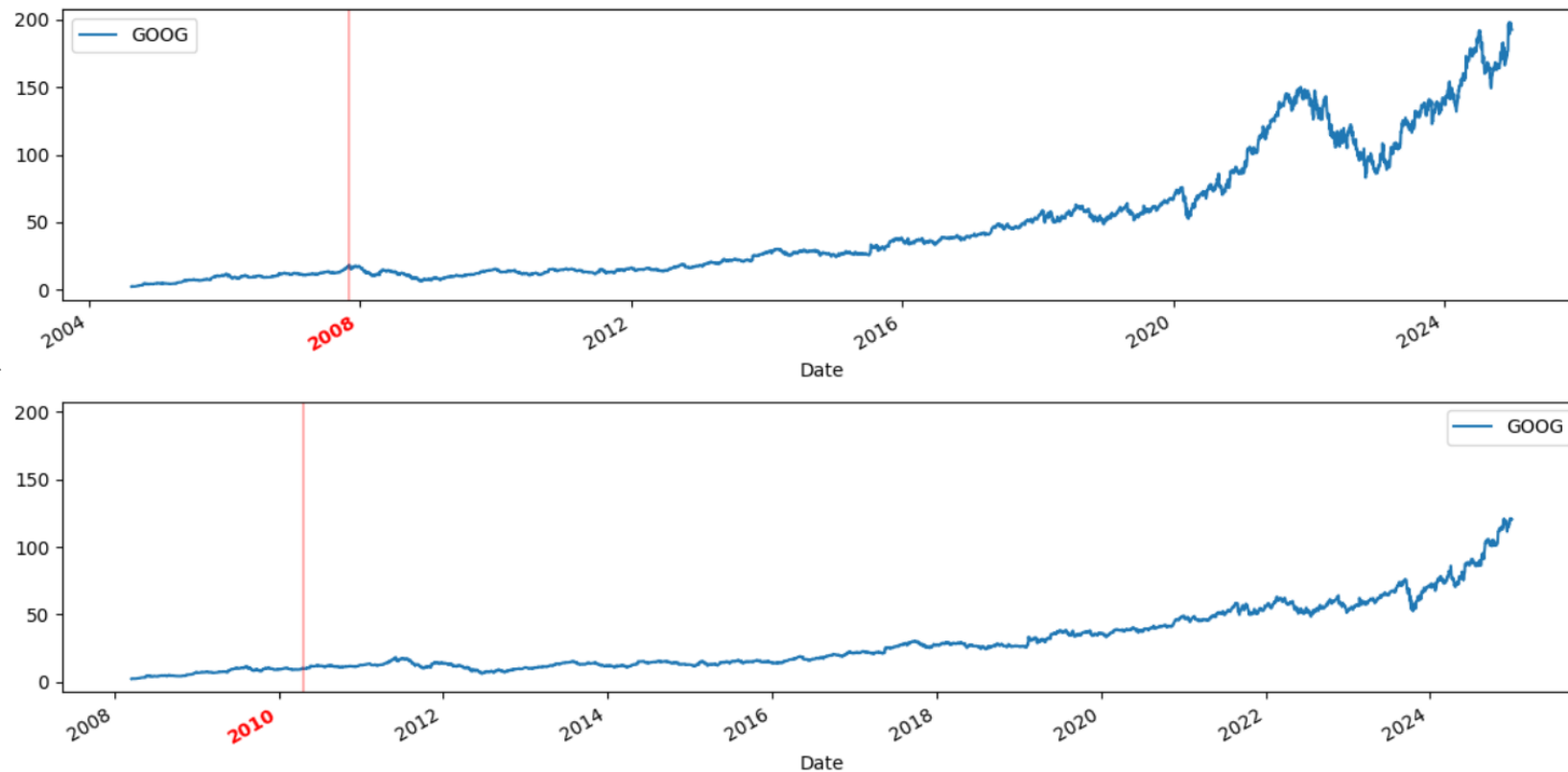


Date	
2004-08-19 00:00:00	2.499133
2004-08-19 12:00:00	NaN
2004-08-20 00:00:00	2.697639
2004-08-20 12:00:00	NaN
2004-08-21 00:00:00	NaN
...	
2022-12-28 00:00:00	86.459999
2022-12-28 12:00:00	NaN
2022-12-29 00:00:00	88.949997
2022-12-29 12:00:00	NaN
2022-12-30 00:00:00	88.730003

Time-shifts

- 시프팅: 시계열을 시간에 따라 이동하는 연산
 - 응용 예: 시차 조정, 변화율 또는 차분 계산
 - 메서드: `shift()`

```
goog.shift(periods=900)
```



Time-Lagged Indicators

- **시차** Time Lag 가 존재하는 두 시계열
 - 시차를 구한 뒤, 시차 만큼 시프팅을 적용
 - 예: 경제 지표 사이의 시차
 - **핵심 물가 상승률** Core Inflation (선행지표)
 - **중소기업 가격 인상 계획** (후행지표)



Time-shifts: Rate of Change

- 변화율 계산

```
goog_shifted = goog.shift(periods=1)  
(goog - goog_shifted) / goog_shifted
```

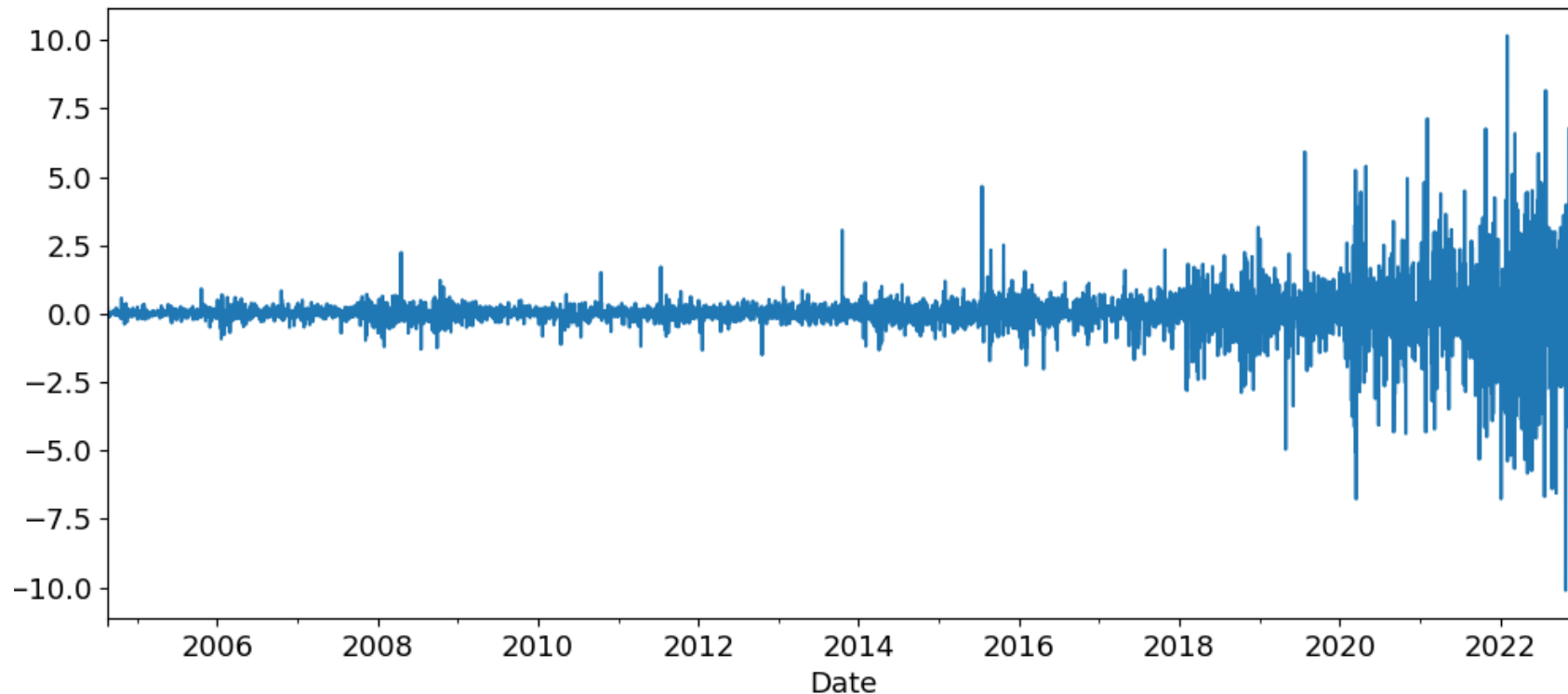


Date	
2004-08-19	NaN
2004-08-20	0.079430
2004-08-21	0.000000
2004-08-22	0.000000
2004-08-23	0.010064
	...
2022-12-26	0.000000
2022-12-27	-0.020933
2022-12-28	-0.016718
2022-12-29	0.028799
2022-12-30	-0.002473

Time-shifts: Differencing

- 차분 Differencing : 현재 값과 이전 값의 차이를 계산
 - 시계열 안정화에 사용됨

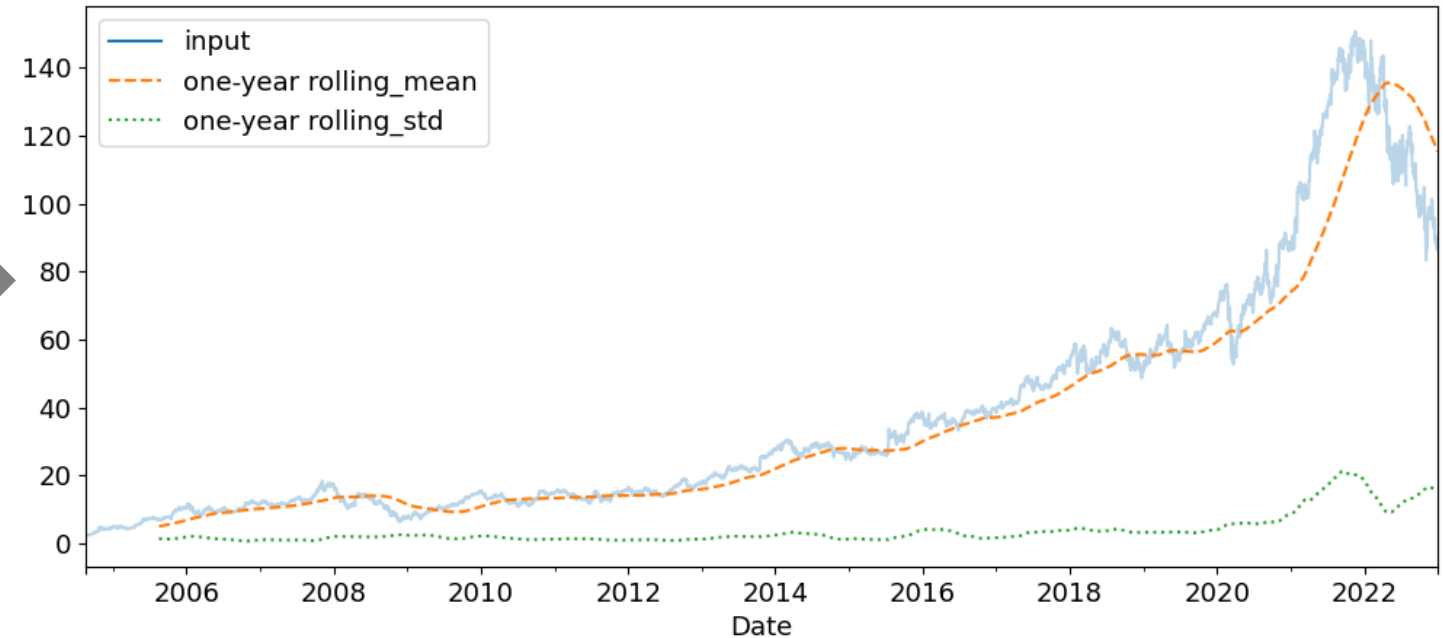
```
goog_diff = goog - goog_shifted
```



Rolling Windows

- 윈도우: 시계열에 대한 집계 연산
 - 이동평균 Moving Average 등을 구할 때 유용

```
goog_rolled = goog.rolling(365)
```



Formatting Datetime: Practical Case

- 타임스탬프 **포매팅/파싱**: 시계열 전처리에서 자주 수행하는 태스크
 - 서로 다른 포맷의 시계열 데이터를 분석할 경우

```
# parset_one
input_file = './data/AirQualityUCI_refined.csv'
input_file = './data/AirQualityUCI_MICE.csv'
input_file = './data/air_co.csv'
input_file = './data/gecco2015.csv'
input_file = './data/gecco2015-2.csv'
input_file = './data/gecco2015-3.csv'

# parser_two
input_file = './data/gecco2015_refined.csv'
input_file = './data/cnnpred_nasdaq_refined.csv'
```

ex) 2004-03-10
18:00:00

ex) 2009-12-31

Formatting Datetime: Practical Case

- 각 포맷에 대한 파서를 `strptime()` 이용하여 함수를 정의

```
def parser_one(x):  
    return datetime.strptime(x, '%Y-%m-%d %H:%M:%S')  
  
def parser_two(x):  
    return datetime.strptime(x, '%Y-%m-%d')
```

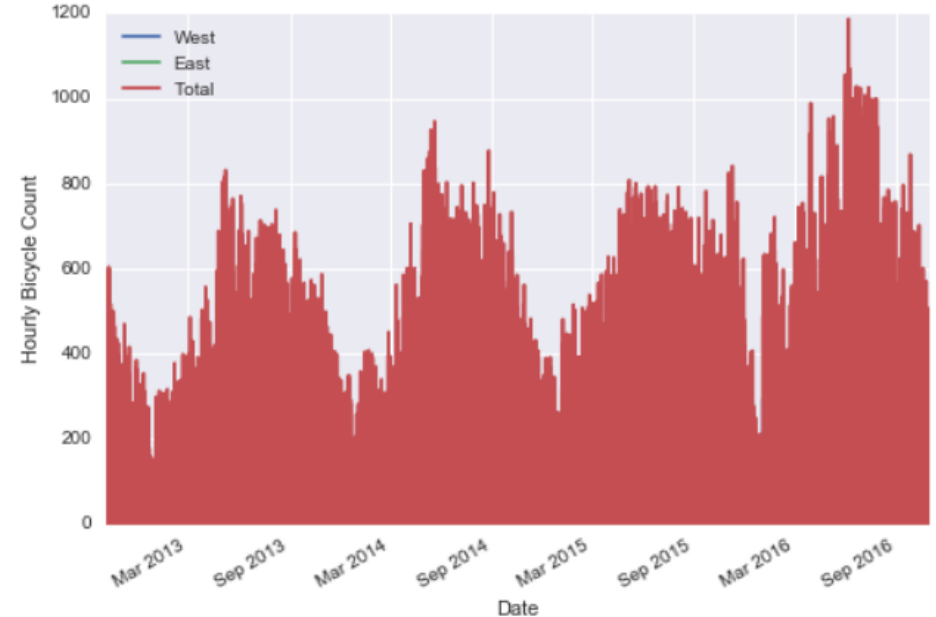
- 시계열 데이터 로딩 때 파서 함수 설정

```
df = pd.read_csv(input_file,  
                 index_col=[0],  
                 parse_dates=[0],  
                 date_parser=parser_one)
```

Additional Learning Materials



- pandas 온라인 문서 ('Time Series/Date' 파트) ([link](#))
- 웨스 맥키니, [파이썬 라이브러리를 활용한 데이터 분석](#), 한빛미디어
 - 비즈니스/재무 환경에서의 시계열 처리 실무에 대한 풍부한 내용 제공
 - 업무 달력 및 표준 시간대 설정 등
- 예제: 시애틀 자전거 시계열 분석 ([link](#))



References



- `pandas.tseries.offsets.DateOffset`, pandas API reference ([link](#))
- VanderPlas, J., *Python Data Science Handbook*, GitHub ([link](#))
- Programiz, *Python strftime()* ([link](#))
- Time series / date functionality, pandas User Guide ([link](#))
- 김정인, 파이썬 데이터 사이언스 핸드북, 위키북스, 2017.



수고하셨습니다.